SE 1105 - INTRODUCTION TO PROGRAMMING AND PROBLEM SOLVING I

Anatomy of a C Program

Asst. Prof. Dr. Kazım Erdoğdu

Faculty of Engineering
Department of Software Engineering
Yaşar University

Fall, 2024

Table of Contents

1. Functions
2. Statements
3. Format Specifiers and Escape Characters
4. Variables
5. Operators
6. Expressions
7. Back to the Functions
8. Exercise

Table of Contents

1. Functions
2. Statements
3. Format Specifiers and Escape Characters
4. Variables
5. Operators
6. Expressions
7. Back to the Functions
8. Exercise

Functions

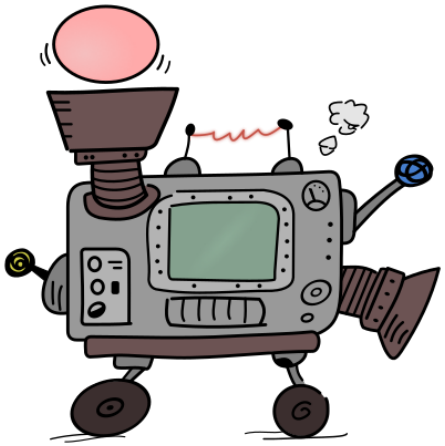

Figure: https://openclipart.org/detail/339619/function-machine

C programs are based on **functions**. Functions in C

- are similar to machines or mathematical functions.
- may have zero or any number of inputs.
- may have zero or one output.
- can be categorized as built-in (e.g. **printf( )**) or user-defined.

HelloWorld.c

```c
#include <stdio.h>
void main( )
{
    printf("Hello World!");
}
```

Sum of Two Integer Numbers

```c
int sum(int a, int b)
{
    return a + b;
}
```

Table of Contents

1. Functions
2. Statements
3. Format Specifiers and Escape Characters
4. Variables
5. Operators
6. Expressions
7. Back to the Functions
8. Exercise

Statements

- **Statements** are instructions that accomplish specific tasks.
- They end with a semicolon (;).

```
1  void main( )
2  {
3      int birth;
4      printf("Enter your year of birth:");
5      scanf("%d",&birth);
6      int age = 2024 - birth;
7      printf("You are %d years old.",age);
8  }
```

Table of Contents

1. Functions
2. Statements
3. Format Specifiers and Escape Characters
4. Variables
5. Operators
6. Expressions
7. Back to the Functions
8. Exercise

Format Specifiers and Escape Characters

Format specifiers[1] in C

- are used to tell the compiler about the type of data to be printed or scanned in input and output operations.
- always start with a % symbol
- are used in the formatted string in functions like printf(), scanf, etc.

Format Specifier	Description
%d	for integer types
%c	for char types
%s	for string types
%f	for float types
%lf	for double types
%e	for scientific (i.e. exponent) notation of floats
%x	for hexadecimal representation
%p	for pointer addresses

Format Specifiers and Escape Characters

Escape characters[2] in C

- are used to represent the characters that cannot be used normally using the keyboard.
- always begins with a \ symbol
- are used in string literals and the formatted string in functions like printf(), scanf, etc.

Escape Character	Description
\n	new line
\t	horizontal tab
\a	beep sound
\'	single quotation mark
\"	double quotation mark
\\	backslash

[1] https://www.geeksforgeeks.org/format-specifiers-in-c/
[2] https://www.geeksforgeeks.org/escape-sequence-in-c/

Table of Contents

1. Functions
2. Statements
3. Format Specifiers and Escape Characters
4. Variables
5. Operators
6. Expressions
7. Back to the Functions
8. Exercise

Variables

Variables are placeholders for unknown quantities.

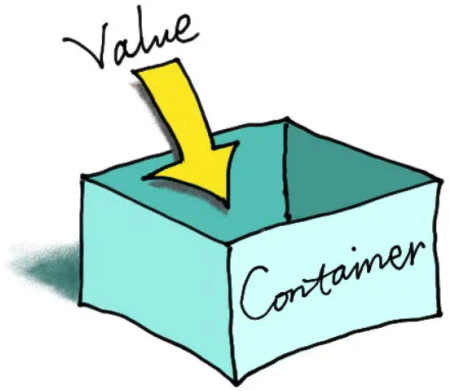

Figure: https://medium.com/@vicmanthebest/comprehending-variables-statement-and-expression-in-javascript-7361094264a4

$$z = 3x^7 - 5xy^2 + 14$$

Variables: x, y, z

Constants: $-5, 2, 3, 7, 14$

Defining Variables in C:

$\langle$**type_of_variable**$\rangle$ $\langle$**name_of_variable**$\rangle$;

Examples:

```
int age;
bool check;
float height;
char c;
```

Basic Variable Types in C

- int, bool, float, double, char, etc.
- We will first focus on **int** and **bool**.
- We can learn the size of a variable type with **sizeof()** function in C.

Type	**Storage Size**	**Value Range**	**Format Specifier**
bool	1 byte	0 or 1	%d
int	4 bytes	–2,147,483,648 to 2,147,483,647	%d

Booleans

- Type name is **bool**.
- Not a built-in data type in C. The relevant header file (i.e. ⟨ stdbool.h ⟩) must be included.
- They use only 1 byte in memory.
- They have only two values: **true** (i.e. 1) or **false** (i.e. 0)
- Format specifier: **%d**
- They are usually used for checking or comparing purposes.

```
bool check = true;
```

Memory Representation:

0x00adfd17								
0x00adfd18								
0x00adfd19								
0x00adfd19	0	0	0	0	0	0	0	1
0x00adfd1A								
0x00adfd1B								
0x00adfd1C								

Integers

- Type name is **int**.
- It is a built-in type. No need a header file.
- They use 4 bytes in memory.
- INT_MIN is –2,147,483,648
- INT_MAX is 2,147,483,647
- Format specifier: **%d**
- How are the negative numbers represented? (2-complement)! (https://www.youtube.com/watch?v=sJXTo3EZoxM)

```
int year = 2024;
```

Memory Representation:

0x00adfd17								
0x00adfd18	0	0	0	0	0	0	0	0
0x00adfd19	0	0	0	0	0	0	0	0
0x00adfd19	0	0	0	0	0	1	1	1
0x00adfd1A	1	1	1	0	1	0	0	0
0x00adfd1B								
0x00adfd1C								

Variable Name Conventions

Variable names

- cannot be a built-in type or function name.
- can begin with any letter or underscore.
- can contain any number of letters, digits, dashes, or underscores.
- cannot contain special and whitespace characters.
- are case-sensitive.

Valid Variable Names	Invalid Variable Names
`int year;`	`int 1year;`
`int Year;`	`int printf;`
`int year_of_birth;`	`int year of birth;`
`int yearOfBirth;`	`int +yearOfBirth;`
`int age5;`	`int (age5;`
`int _weight;`	`int -weight;`

Table of Contents

1. Functions
2. Statements
3. Format Specifiers and Escape Characters
4. Variables
5. Operators
6. Expressions
7. Back to the Functions
8. Exercise

Operators

They are symbols that help programs to perform some mathematical, relational, conditional, logical, or bitwise computations on variables and their values.[3]

- Assignment Operator: $=$
- Arithmetic Operators: $+$, $-$, $*$, $/$, $\%$
- Increment/Decrement Operators: $++$, $--$
- Combined Assignment Operators: $+=$, $-=$, $*=$, $/=$, $\%=$
- Rational Operators: $==$, $!=$, $<$, $<=$, $>$, $>=$
- Logical Operators: $\&\&$, $||$, $!$
- Bitwise Operators: $\&$, $|$, $<<$, $>>$, $\ ,$
- Ternary Operator: $?:$

Operators

Operators in C

```c
#include <stdio.h>
void main( )
{
    int a,b=10,c;
    c = 3;
    a = b + c;
    printf("%d\n",a);
    printf("%d\n",b / c);
    printf("%d\n",b % c);
    a++;
    printf("%d\n",a);
    a = a - 3;
    printf("%d\n",a <= b);
    printf("%d\n",a==b || c != b);
}
```

[3] https://www.geeksforgeeks.org/operators-in-c/

Table of Contents

1. Functions
2. Statements
3. Format Specifiers and Escape Characters
4. Variables
5. Operators
6. Expressions
7. Back to the Functions
8. Exercise

Expressions

- Anything that has a value is called an expression.
- Type of the expression is determined by its value.
- Examples:
  - Immediate values
  - Variables
  - Operators
  - Function calls

Table of Contents

1. Functions
2. Statements
3. Format Specifiers and Escape Characters
4. Variables
5. Operators
6. Expressions
7. Back to the Functions
8. Exercise

Defining a Function

type function_name (parameters)
{
Statements and Expressions
[Optional] **return** statement
}

Functions in C

```
#include <stdio.h>

int square(int a)
{
    return a*a;
}
```

Calling a Function

Functions in C

```
1   #include <stdio.h>
2
3   int square(int a)
4   {
5       return a*a;
6   }
7
8   void main(){
9       int n = 4;
10      int p = square(n);
11      printf("Square of %d is %d\n",n,p);
12  }
```

Table of Contents

1. Functions
2. Statements
3. Format Specifiers and Escape Characters
4. Variables
5. Operators
6. Expressions
7. Back to the Functions
8. Exercise

Exercise

- Write a function that prints "Hello" to the screen
- Write a function that prints 3 times "Hello" to the screen.
- Write a function that takes 3 integers as parameters and returns their sum.
- Write a function that takes a 4-digit number and prints its digits separately in each line on the screen.